

HarborSync

Teknik Sistem Raporu

Versiyon: 2.0 | Tarih: 13 Mayıs 2026

Hazırlayan: Emirhan Ersoy & Deniz Yıldız

Ders: CENG-442 Microservice Architecture

İÇİNDEKİLER

- Yönetici Özeti
- Sisteme Genel Bakış
- Mimari Genel Bakış
- Servisler ve Modüller
- API Sözleşmeleri ve Olay Şemaları
- Veritabanı Mimarisi
- Frontend Mimarisi
- DevOps ve Altyapı
- Güvenlik Mimarisi
- Gözlemlenebilirlik ve İzleme
- Riskler ve Teknik Borç
- İyileştirme Önerileri
- Test Stratejisi
- Geliştirme Takvimi
- Ekler

1. Yönetici Özeti

HarborSync, liman sahasını izleyen drone simülatörlerinden gelen anlık telemetri verilerini işleyerek konteyner sektörlerindeki operasyonel yoğunluğu analiz eden ve bu analizlere dayanarak vinç ve saha ekiplerine otomatik görev atamaları gerçekleştiren dağıtık bir kargo koordinasyon platformudur.

Sistem, mikroservis mimarisi temelinde geliştirilmiş olup her bir servis belirli bir iş alanından sorumludur. Servisler arası iletişim büyük ölçüde RabbitMQ mesajlaşma sistemi üzerinden asenkron olarak gerçekleştirilmektedir. "Database per Service" prensibi benimsenerek servisler arası bağımlılık en aza indirilmiş, "Eventual Consistency" modeli ile dağıtık tutarlılık sağlanmıştır.

Servis	Teknoloji	Sorumlu	Port
API Gateway	Spring Cloud Gateway	Emirhan	8080
Vessel Service	Spring Boot 3 + PostgreSQL	Emirhan	8081
Telemetry Service	Spring Boot 3 + Redis	Deniz	8082
Congestion Analysis	Spring Boot 3 (stateless)	Emirhan	8083
Task Assignment	Spring Boot 3 + PostgreSQL	Deniz	8084
Notification Service	FastAPI (Python 3.11)	Deniz	8085

2. Sisteme Genel Bakış

HarborSync, liman operasyonlarının verimliliğini artırmak amacıyla tasarlanmış olay güdümlü bir mikroservis platformudur. Drone'lardan gelen anlık saha verilerini kullanarak konteyner alanlarındaki yoğunluğu tespit eder ve bu yoğunluğu azaltmak için proaktif görevler oluşturur.

Temel İş Akışı

- Veri Toplama:** Drone simülatörleri telemetri verilerini RabbitMQ üzerinden sisteme gönderir.
- Veri İşleme:** Telemetry Service ham verileri alır, doğrular, zenginleştirir ve `telemetry.processed` kuyruğuna yayınlar.
- Analiz:** Congestion Analysis Service işlenmiş verileri dinler; eşik aşılsa `CongestionAlertEvent` üretir.
- Görev Atama:** Task Assignment Service alarmları dinler, Vessel Service'i sorgular ve görev oluşturur.
- Bildirim:** Notification Service tüm önemli olayları yapılandırılmış log dosyasına yazar.

Proje Dizin Yapısı

```

harborsync/
├─ docker-compose.yml
├─ .env
├─ README.md
├─ gateway/                ← Emirhan
├─ vessel-service/         ← Emirhan
├─ congestion-analysis/    ← Emirhan
├─ telemetry-service/      ← Deniz
├─ task-assignment-service/ ← Deniz
├─ notification-service/   ← Deniz
└─ drone-simulator/        ← Deniz (Python script)

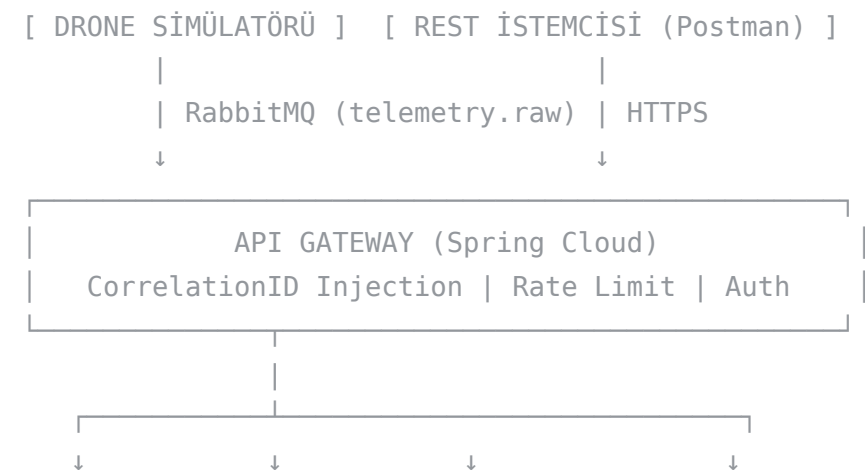
```

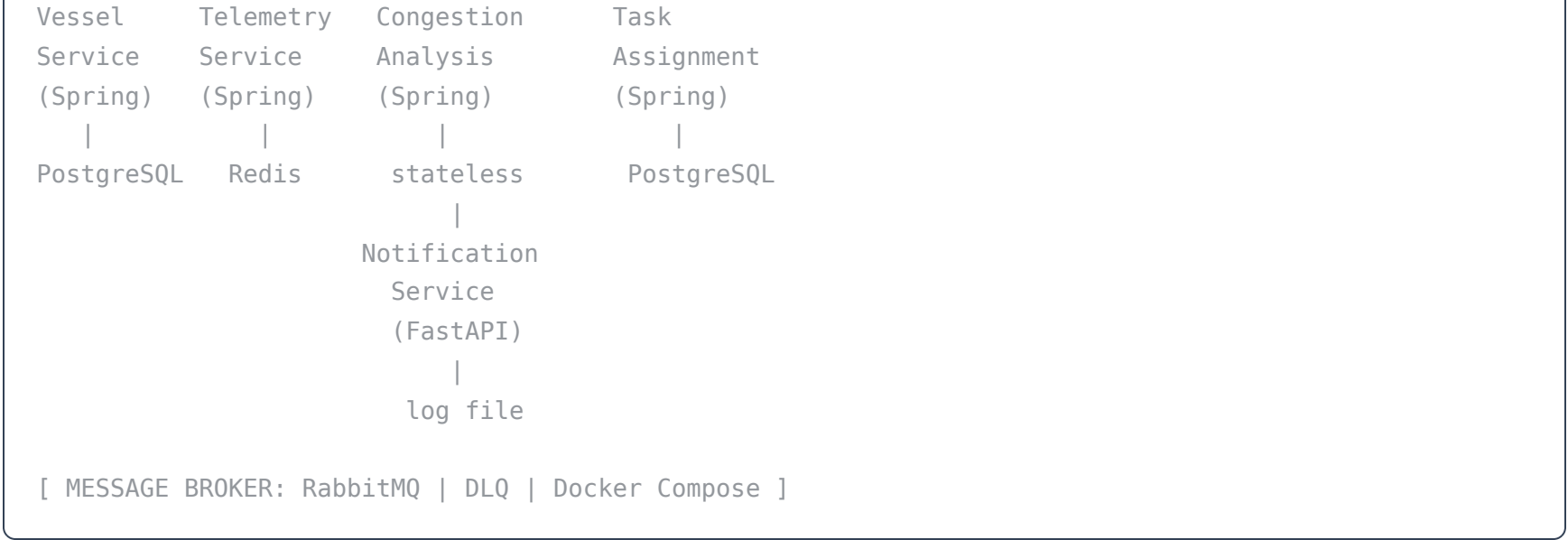
Spring Boot Servis İç Paket Yapısı

```
com.harborsync.<servisadi>/
├─ controller/           # REST endpoint'leri (varsa)
├─ service/              # İş mantığı
├─ repository/           # JPA/Redis repository'leri
├─ domain/               # Entity sınıfları
├─ dto/                  # Request/Response DTO'ları
├─ messaging/
│   ├─ consumer/         # RabbitMQ listener'ları
│   └─ producer/         # RabbitMQ publisher'ları
├─ config/               # RabbitMQ, Redis, vb. konfigürasyonlar
└─ exception/            # Custom exception sınıfları
```

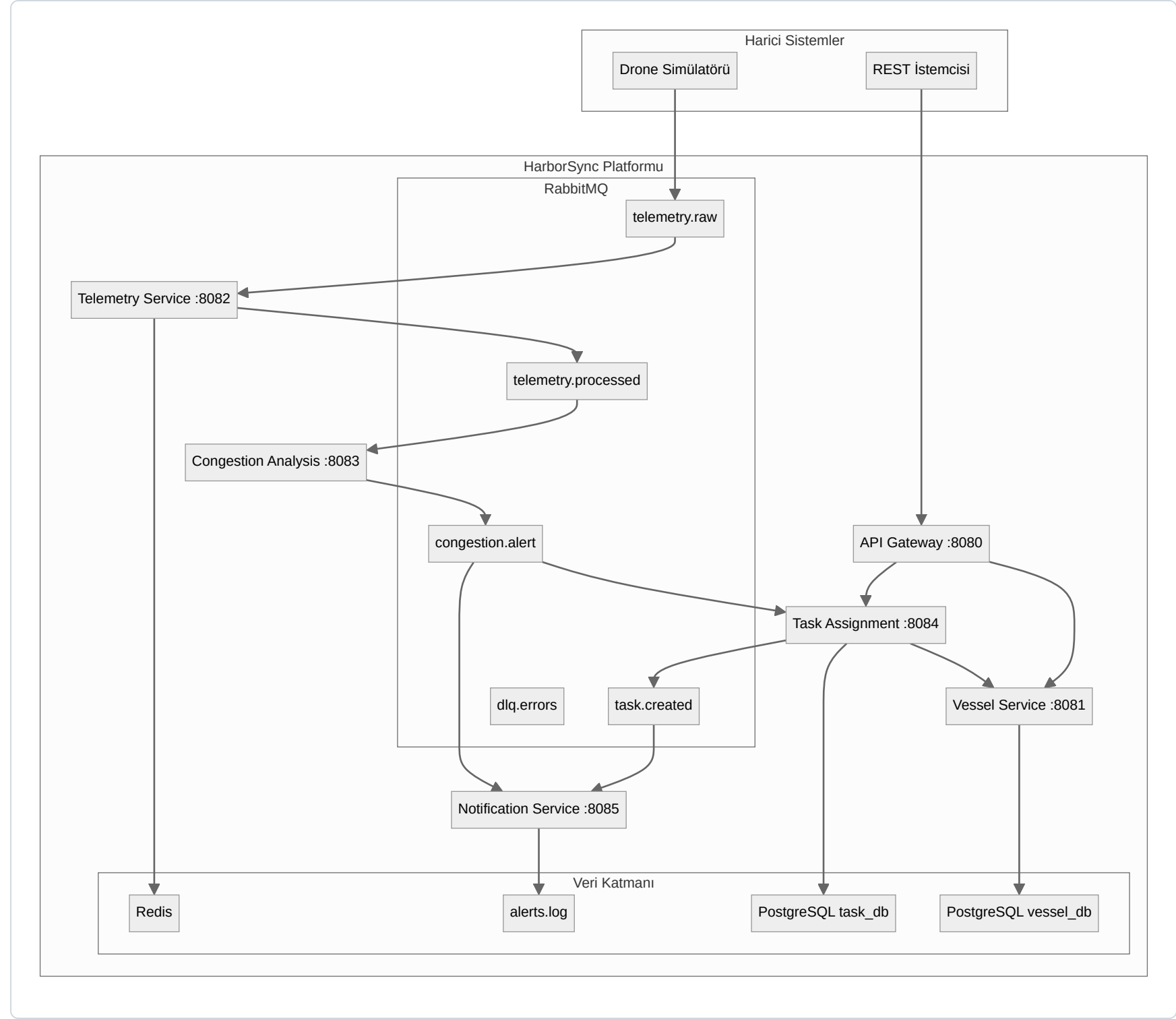
3. Mimari Genel Bakış

ASCII Mimari Şeması





Yüksek Seviye Mimari Diyagramı



Tasarım Kararları

Karar	Seçim	Gerekçe
Mesajlaşma	RabbitMQ	Kafka'ya göre düşük operasyonel karmaşıklık, DLQ desteği
Servis izolasyonu	Database per Service	Servis sınırlarını güçlendirir, SPOF önler
Tutarlılık modeli	Eventual Consistency	Dağıtık sistemde distributed transaction gereksiz
Telemetry işleme	Stateless	Yatay ölçekleme kolaylığı

Karar	Seçim	Gerekçe
Frontend	Yok	Sadece REST API + structured log çıktısı

RabbitMQ Exchange ve Queue Yapısı:

Exchange: `harborsync.exchange` (type: direct)

Kuyruklar: `telemetry.raw` · `telemetry.processed` · `congestion.alert` · `task.created` · `dlq.errors`

Her kuyruğun `x-dead-letter-exchange` argümanı ile DLQ'ya yönlendirilmesi zorunludur.

4. Servisler ve Modüller

4.1 Vessel Service

Sorumlu: Emirhan Ersoy · Port: 8081

Vessel Service , liman sahasına gelen ve ayrılan gemilerin yaşam döngüsünü yöneten merkezi servistir. Sistemin "gemilerle ilgili tek doğru kaynak" (single source of truth) rolünü üstlenir. Task Assignment Service , görev atama mantığı içinde gemi bilgilerine erişmek için bu servise doğrudan senkron REST çağrısı yapar.

Teknoloji Yığını

Katman	Teknoloji	Notlar
Framework	Spring Boot 3.2.x	Ana uygulama çatısı
Veri Erişimi	Spring Data JPA + Hibernate	PostgreSQL ile etkileşim
Veritabanı	PostgreSQL 15	İlişkisel gemi verileri
Migration	Flyway	Şema değişikliklerini versiyonlar
Build	Maven 3.9.x	—
Port	8081	—

Maven pom.xml Bağımlılıkları

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>org.flywaydb</groupId>
    <artifactId>flyway-core</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

application.yml

```
server:
  port: 8081

spring:
  application:
    name: vessel-service
  datasource:
    url: jdbc:postgresql://localhost:5433/vessel_db
    username: harbor
    password: harbor123
    driver-class-name: org.postgresql.Driver
  jpa:
    hibernate:
      ddl-auto: validate          # Flyway yönetir, Hibernate dokunmaz
    show-sql: false
```

```
flyway:
  enabled: true
  locations: classpath:db/migration

logging:
  pattern:
    console: "[%d{HH:mm:ss}] [%-5level] [%X{correlationId}] %logger{36} - %msg%n"
```

Veritabanı Şeması — Flyway Migration

Dosya: src/main/resources/db/migration/V1__create_vessels_table.sql

```
CREATE TABLE vessels (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name        VARCHAR(100) NOT NULL,
  imo_number  VARCHAR(20) UNIQUE NOT NULL,
  status      VARCHAR(20) NOT NULL DEFAULT 'ARRIVING',
  berth       VARCHAR(10),
  eta         TIMESTAMP,
  created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
  updated_at  TIMESTAMP NOT NULL DEFAULT NOW()
);

ALTER TABLE vessels ADD CONSTRAINT chk_vessel_status
  CHECK (status IN ('ARRIVING', 'DOCKED', 'DEPARTING', 'DEPARTED'));

CREATE INDEX idx_vessel_status ON vessels(status);
```

Domain Entity

```
// domain/Vessel.java
@Entity
@Table(name = "vessels")
@Data
@NoArgsConstructor
public class Vessel {

    @Id
    @GeneratedValue
    private UUID id;

    @Column(nullable = false)
    private String name;

    @Column(name = "imo_number", unique = true, nullable = false)
```



```
private String imoNumber;

@Enumerated(EnumType.STRING)
@Column(nullable = false)
private VesselStatus status = VesselStatus.ARRIVING;

private String berth;
private LocalDateTime eta;

@Column(name = "created_at", updatable = false)
private LocalDateTime createdAt = LocalDateTime.now();

@Column(name = "updated_at")
private LocalDateTime updatedAt = LocalDateTime.now();

@PreUpdate
public void preUpdate() { this.updatedAt = LocalDateTime.now(); }
}

// domain/VesselStatus.java
public enum VesselStatus {
    ARRIVING, DOCKED, DEPARTING, DEPARTED
}
```

DTO Sınıfları

```
// dto/CreateVesselRequest.java
@Data
public class CreateVesselRequest {
    @NotBlank private String name;
    @NotBlank private String imoNumber;
    private LocalDateTime eta;
}

// dto/UpdateVesselStatusRequest.java
@Data
public class UpdateVesselStatusRequest {
    @NotNull private VesselStatus status;
    private String berth;
}

// dto/VesselResponse.java
@Data @Builder
public class VesselResponse {
    private UUID id;
    private String name;
    private String imoNumber;
```

```
private VesselStatus status;
private String berth;
private LocalDateTime eta;
}
```

Repository

```
// repository/VesselRepository.java
public interface VesselRepository extends JpaRepository<Vessel, UUID> {
    List<Vessel> findByStatus(VesselStatus status);
    Optional<Vessel> findByImoNumber(String imoNumber);
}
```

Service Katmanı

```
// service/VesselService.java
@Service
@RequiredArgsConstructor
@Slf4j
public class VesselService {

    private final VesselRepository vesselRepository;

    public VesselResponse registerVessel(CreateVesselRequest request) {
        Vessel vessel = new Vessel();
        vessel.setName(request.getName());
        vessel.setImoNumber(request.getImoNumber());
        vessel.setEta(request.getEta());
        vessel.setStatus(VesselStatus.ARRIVING);
        Vessel saved = vesselRepository.save(vessel);
        log.info("Vessel registered: {} ({})", saved.getName(), saved.getId());
        return toResponse(saved);
    }

    public List<VesselResponse> getVesselsByStatus(VesselStatus status) {
        return vesselRepository.findByStatus(status).stream().map(this::toResponse).toList();
    }

    public VesselResponse updateStatus(UUID id, UpdateVesselStatusRequest request) {
        Vessel vessel = vesselRepository.findById(id)
            .orElseThrow(() -> new VesselNotFoundException("Vessel not found: " + id));
        vessel.setStatus(request.getStatus());
        if (request.getBerth() != null) vessel.setBerth(request.getBerth());
        return toResponse(vesselRepository.save(vessel));
    }
}
```

```
private VesselResponse toResponse(Vessel v) {  
    return VesselResponse.builder()  
        .id(v.getId()).name(v.getName()).imoNumber(v.getImoNumber())  
        .status(v.getStatus()).berth(v.getBerth()).eta(v.getEta()).build();  
}  
}
```

Controller

```
// controller/VesselController.java  
@RestController  
@RequestMapping("/vessels")  
@RequiredArgsConstructor  
public class VesselController {  
  
    private final VesselService vesselService;  
  
    @PostMapping  
    @ResponseStatus(HttpStatus.CREATED)  
    public VesselResponse registerVessel(@Valid @RequestBody CreateVesselRequest request) {  
        return vesselService.registerVessel(request);  
    }  
  
    @GetMapping  
    public List<VesselResponse> listVessels(@RequestParam(required = false) VesselStatus status) {  
        if (status != null) return vesselService.getVesselsByStatus(status);  
        return vesselService.getAllVessels();  
    }  
  
    @GetMapping("/{id}")  
    public VesselResponse getVessel(@PathVariable UUID id) {  
        return vesselService.getVesselById(id);  
    }  
  
    @PutMapping("/{id}/status")  
    public VesselResponse updateStatus(@PathVariable UUID id,  
        @Valid @RequestBody UpdateVesselStatusRequest request) {  
        return vesselService.updateStatus(id, request);  
    }  
}
```

Exception Handling

```
// exception/VesselNotFoundException.java
public class VesselNotFoundException extends RuntimeException {
    public VesselNotFoundException(String message) { super(message); }
}

// exception/GlobalExceptionHandler.java
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(VesselNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public Map<String, String> handleNotFound(VesselNotFoundException ex) {
        return Map.of("error", ex.getMessage());
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    @ResponseStatus(HttpStatus.BAD_REQUEST)
    public Map<String, String> handleValidation(MethodArgumentNotValidException ex) {
        return ex.getBindingResult().getFieldErrors().stream()
            .collect(Collectors.toMap(FieldError::getField, FieldError::getDefaultMessage));
    }
}
```

Correlation ID MDC Filtresi

Bu filtre **her Spring Boot servisine** eklenmelidir. Gateway'den gelen X-Correlation-ID header'ını MDC'ye yazar; yoksa yeni bir UUID üretir.

```
// config/CorrelationIdFilter.java
@Component
@Order(1)
public class CorrelationIdFilter implements Filter {

    private static final String CORRELATION_HEADER = "X-Correlation-ID";

    @Override
    public void doFilter(ServletRequest req, ServletResponse res, FilterChain chain)
        throws IOException, ServletException {
        HttpServletRequest httpReq = (HttpServletRequest) req;
        String correlationId = httpReq.getHeader(CORRELATION_HEADER);
        if (correlationId == null) correlationId = UUID.randomUUID().toString();
        MDC.put("correlationId", correlationId);
        try {
            chain.doFilter(req, res);
        }
    }
}
```

```
        } finally {
            MDC.clear();
        }
    }
}
```

API Endpoint'leri

Metot	Yol	Açıklama	Örnek Body
POST	/vessels	Yeni gemi kaydı	<code>{"name": "MV-Ankara", "imoNumber": "IM01234567"}</code>
GET	/vessels? status=ARRIVING	Duruma göre filtreli liste	—
GET	/vessels/{id}	Gemi detayı	—
PUT	/vessels/{id}/status	Durum ve rıhtım güncelle	<code>{"status": "DOCKED", "berth": "A-03"}</code>

4.2 Congestion Analysis Service

Sorumlu: Emirhan Ersoy · Port: 8083

`telemetry.processed` kuyruğunu dinler, önceden tanımlı iş kurallarını değerlendirir ve eşik aşılsa `congestion.alert` kuyruğuna alarm üretir. **Veritabanı yoktur; tamamen stateless çalışır.**

Teknoloji Yığını

Katman	Teknoloji	Notlar
Framework	Spring Boot 3.2.x	Ana uygulama çatısı
Mesajlaşma	Spring AMQP (RabbitMQ)	Consumer + Producer
İş Mantığı	Dahili Kural Motoru	<code>CongestionRuleEngine.java</code>
Port	8083	—

Maven pom.xml Bağımlılıkları

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
    <!-- Sadece actuator/health endpoint için -->
  </dependency>
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

application.yml

```
server:
  port: 8083

spring:
  application:
    name: congestion-analysis
  rabbitmq:
    host: localhost
    port: 5672
    username: guest
    password: guest

  harborsync:
    queues:
      telemetry-processed: telemetry.processed
      congestion-alert: congestion.alert
  thresholds:
    critical: 0.90
    warning: 0.85
```

RabbitMQ Konfigürasyonu

```
// config/RabbitMQConfig.java
@Configuration
public class RabbitMQConfig {

    public static final String EXCHANGE = "harborsync.exchange";
    public static final String TELEMETRY_PROCESSED_QUEUE = "telemetry.processed";
    public static final String CONGESTION_ALERT_QUEUE = "congestion.alert";
    public static final String DLQ = "dlq.errors";

    @Bean public DirectExchange harborSyncExchange() {
        return new DirectExchange(EXCHANGE);
    }

    @Bean public Queue telemetryProcessedQueue() {
        return QueueBuilder.durable(TELEMETRY_PROCESSED_QUEUE)
            .withArgument("x-dead-letter-exchange", "")
            .withArgument("x-dead-letter-routing-key", DLQ)
            .build();
    }

    @Bean public Queue congestionAlertQueue() {
        return QueueBuilder.durable(CONGESTION_ALERT_QUEUE)
            .withArgument("x-dead-letter-exchange", "")
            .withArgument("x-dead-letter-routing-key", DLQ)
            .build();
    }

    @Bean public Queue deadLetterQueue() {
        return QueueBuilder.durable(DLQ).build();
    }

    @Bean public Binding telemetryBinding(Queue telemetryProcessedQueue, DirectExchange exchange) {
        return BindingBuilder.bind(telemetryProcessedQueue).to(exchange).with(TELEMETRY_PROCESSED_QUEUE);
    }

    @Bean public Binding congestionBinding(Queue congestionAlertQueue, DirectExchange exchange) {
        return BindingBuilder.bind(congestionAlertQueue).to(exchange).with(CONGESTION_ALERT_QUEUE);
    }

    @Bean public MessageConverter messageConverter() {
        return new Jackson2JsonMessageConverter();
    }

    @Bean public RabbitTemplate rabbitTemplate(ConnectionFactory connectionFactory) {
        RabbitTemplate template = new RabbitTemplate(connectionFactory);
        template.setMessageConverter(messageConverter());
    }
}
```

```
        return template;
    }
}
```

Event DTO'ları

```
// dto/ProcessedTelemetryEvent.java
@Data @NoArgsConstructor @AllArgsConstructor
public class ProcessedTelemetryEvent {
    private String correlationId;
    private String sector;
    private double fillRate;
    private boolean blockageDetected;
    private String droneId;
    private String vesselEta;    // null ise yaklaşan gemi yok
    private String timestamp;
}

// dto/CongestionAlertEvent.java
@Data @Builder
public class CongestionAlertEvent {
    private String correlationId;
    private String alertType;    // SECTOR_CRITICAL | SECTOR_WARNING | IMMEDIATE_ACTION
    private String sector;
    private String severity;    // HIGH | MEDIUM | LOW
    private double fillRate;
    private String recommendedAction;
    private String timestamp;
}
```

Kural Motoru (Core Logic)

```
// service/CongestionRuleEngine.java
@Service
@Slf4j
public class CongestionRuleEngine {

    @Value("${harborsync.thresholds.critical}")
    private double criticalThreshold;

    @Value("${harborsync.thresholds.warning}")
    private double warningThreshold;

    /**
```



```
* Telemetri event'ine karşılık gelen alarm tipini hesaplar.
* Kural motoru tamamen stateless; tüm kararlar anlık veriye dayanır.
*/
public Optional<CongestionAlertEvent> evaluate(ProcessedTelemetryEvent event) {

    // Kural 1: Tıkanıklık + yaklaşan gemi → IMMEDIATE_ACTION (en yüksek öncelik)
    if (event.isBlockageDetected() && event.getVesselEta() != null) {
        return Optional.of(buildAlert(event, "IMMEDIATE_ACTION", "HIGH", "HOLD_VESSEL"));
    }

    // Kural 2: Doluluk > %90 → SECTOR_CRITICAL
    if (event.getFillRate() > criticalThreshold) {
        return Optional.of(buildAlert(event, "SECTOR_CRITICAL", "HIGH", "REDIRECT_CRANE"));
    }

    // Kural 3: Doluluk > %85 → SECTOR_WARNING
    if (event.getFillRate() > warningThreshold) {
        return Optional.of(buildAlert(event, "SECTOR_WARNING", "MEDIUM", "MONITOR_SECTOR"));
    }

    log.debug("Sector {} is nominal (fillRate={})", event.getSector(), event.getFillRate());
    return Optional.empty();
}

private CongestionAlertEvent buildAlert(ProcessedTelemetryEvent event,
                                       String alertType, String severity,
                                       String recommendedAction) {
    log.warn("[{}] ALERT: {} for sector {} (fillRate={})",
            event.getCorrelationId(), alertType, event.getSector(), event.getFillRate());
    return CongestionAlertEvent.builder()
        .correlationId(event.getCorrelationId())
        .alertType(alertType).sector(event.getSector())
        .severity(severity).fillRate(event.getFillRate())
        .recommendedAction(recommendedAction)
        .timestamp(Instant.now().toString())
        .build();
}
}
```

RabbitMQ Consumer & Producer

```
// messaging/consumer/TelemetryConsumer.java
@Component
@RequiredArgsConstructor
@Slf4j
public class TelemetryConsumer {
```

```
private final CongestionRuleEngine ruleEngine;
private final CongestionAlertProducer alertProducer;

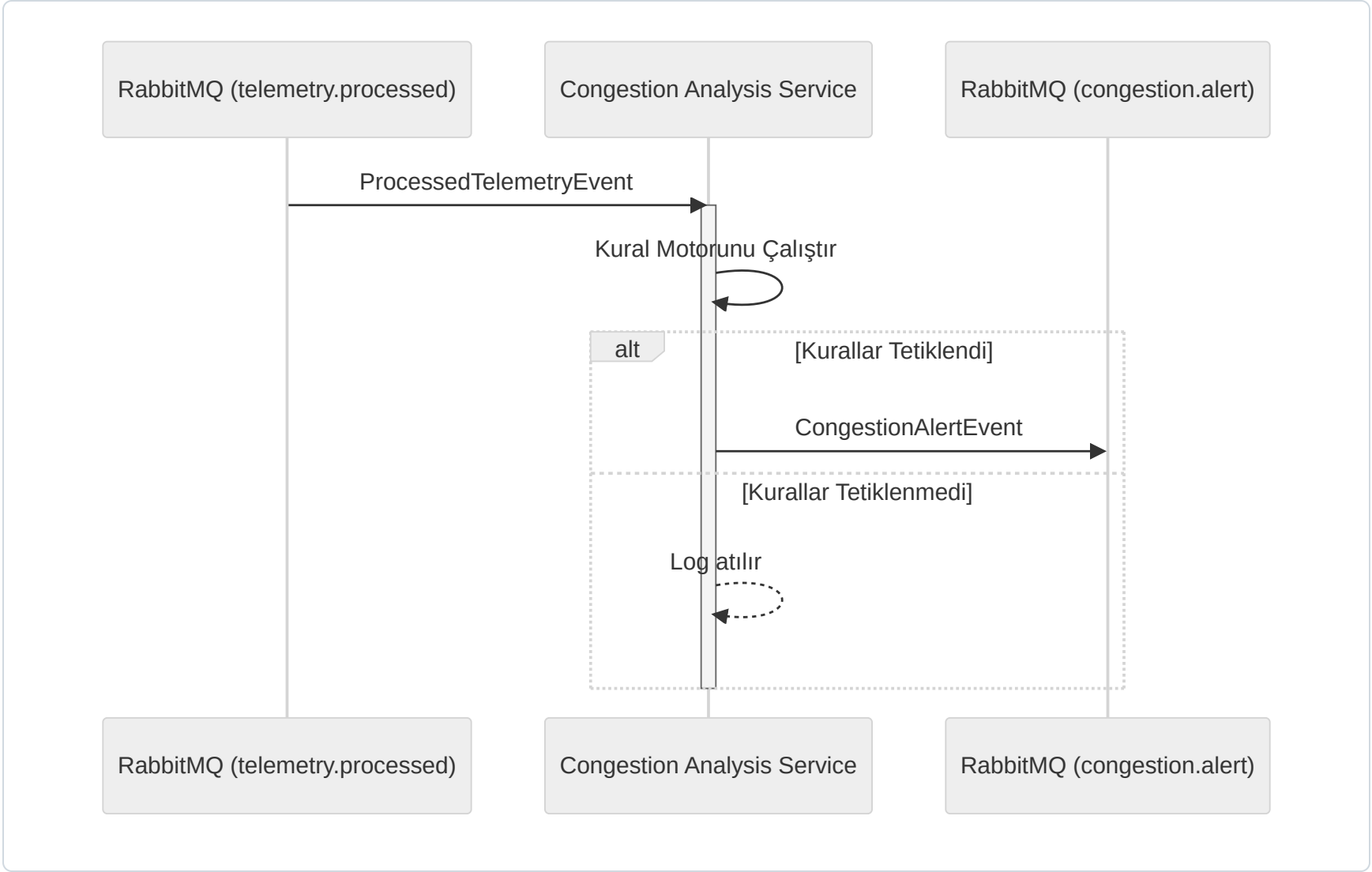
    @RabbitListener(queues = RabbitMQConfig.TELEMETRY_PROCESSED_QUEUE)
    public void onTelemetryReceived(ProcessedTelemetryEvent event) {
        MDC.put("correlationId", event.getCorrelationId());
        try {
            log.info("Received telemetry for sector {} (fillRate={})",
                event.getSector(), event.getFillRate());
            ruleEngine.evaluate(event).ifPresent(alertProducer::publishAlert);
        } finally {
            MDC.clear();
        }
    }
}

// messaging/producer/CongestionAlertProducer.java
@Component
@RequiredArgsConstructor
@Slf4j
public class CongestionAlertProducer {

    private final RabbitTemplate rabbitTemplate;

    public void publishAlert(CongestionAlertEvent alert) {
        rabbitTemplate.convertAndSend(
            RabbitMQConfig.EXCHANGE, RabbitMQConfig.CONGESTION_ALERT_QUEUE, alert);
        log.info("[{}] Alert published: {} for sector {}",
            alert.getCorrelationId(), alert.getAlertType(), alert.getSector());
    }
}
```

İş Akışı Diyagramı



4.3 API Gateway

Sorumlu: Emirhan Ersoy · Port: 8080

Sisteme giren tüm REST trafiğinin tek giriş noktası. Correlation ID üretimi, rate limiting ve downstream servislere yönlendirme sorumluluklarını üstlenir.

Teknoloji Yığını

Katman	Teknoloji
Framework	Spring Cloud Gateway 4.x
Reaktif model	Project Reactor (WebFlux)
Port	8080

Maven pom.xml Bağımlılıkları

```
<dependencies>
  <dependency>
    <groupId>org.springframework.cloud</groupId>
    <artifactId>spring-cloud-starter-gateway</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-actuator</artifactId>
  </dependency>
</dependencies>

<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework.cloud</groupId>
      <artifactId>spring-cloud-dependencies</artifactId>
      <version>2023.0.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

application.yml

```
server:
  port: 8080

spring:
  application:
    name: api-gateway
  cloud:
    gateway:
      routes:
        - id: vessel-service
          uri: http://vessel-service:8081
          predicates:
            - Path=/api/vessels/**
      filters:
        - StripPrefix=1 # /api prefix'ini soyar
        - name: RequestRateLimiter
          args:
            redis-rate-limiter.replenishRate: 10
            redis-rate-limiter.burstCapacity: 20
```

```
- id: task-service
uri: http://task-assignment-service:8084
predicates:
  - Path=/api/tasks/**
filters:
  - StripPrefix=1

default-filters:
  - name: CorrelationIdFilter    # Custom filter – aşağıda tanımlı
```

Correlation ID Global Filter

```
// filter/CorrelationIdGatewayFilter.java
@Component
public class CorrelationIdGatewayFilter implements GlobalFilter, Ordered {

    private static final String CORRELATION_HEADER = "X-Correlation-ID";

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String correlationId = exchange.getRequest().getHeaders().getFirst(CORRELATION_HEADER);

        if (correlationId == null || correlationId.isBlank()) {
            correlationId = "OP-" + UUID.randomUUID().toString().substring(0, 8).toUpperCase();
        }

        final String finalCorrelationId = correlationId;
        ServerWebExchange mutatedExchange = exchange.mutate()
            .request(req -> req.header(CORRELATION_HEADER, finalCorrelationId))
            .response(res -> res.getHeaders().add(CORRELATION_HEADER, finalCorrelationId))
            .build();

        return chain.filter(mutatedExchange);
    }

    @Override
    public int getOrder() { return -1; }    // En yüksek öncelik
}
```

Logging Filter

```
// filter/RequestLoggingFilter.java
@Component
@Slf4j
```



```
public class RequestLoggingFilter implements GlobalFilter, Ordered {

    @Override
    public Mono<Void> filter(ServerWebExchange exchange, GatewayFilterChain chain) {
        String correlationId = exchange.getRequest().getHeaders().getFirst("X-Correlation-ID");
        String method = exchange.getRequest().getMethod().name();
        String path = exchange.getRequest().getPath().toString();
        log.info("[Gateway] {} {} correlationId={}", method, path, correlationId);
        return chain.filter(exchange);
    }

    @Override
    public int getOrder() { return 0; }
}
```

Correlation ID Mekanizması: Gateway, gelen istekte X-Correlation-ID yoksa "OP-" önekiyle yeni bir tane üretir. Bu ID hem yanıtı eklenir hem de aşağı akıştaki tüm servislere iletilir. Böylece bir isteğin sistem içindeki tüm yolculuğu tek bir ID üzerinden izlenebilir.

4.4 Telemetry Service

Sorumlu: Deniz Yıldız · Port: 8082

Drone simülatörlerinden ham telemetri alır, doğrular, son drone state'ini Redis'e yazar ve işlenmiş event'i telemetry.processed kuyruğuna basar. Sistemin "duyu organı" olarak görev yapar.

Teknoloji Yığını

Katman	Teknoloji	Notlar
Framework	Spring Boot 3.2.x	Ana uygulama çatısı
Cache / Durum Deposu	Spring Data Redis (Lettuce)	Drone durumunu TTL:30s ile saklar
Mesajlaşma	Spring AMQP	İşlenmiş olayları RabbitMQ'ya yayınlar
Port	8082	—

Maven pom.xml Bağımlılıkları

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-validation</artifactId>
  </dependency>
  <dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
  </dependency>
</dependencies>
```

application.yml

```
server:
  port: 8082

spring:
  application:
    name: telemetry-service
  data:
    redis:
      host: localhost
      port: 6379
  rabbitmq:
    host: localhost
    port: 5672
    username: guest
    password: guest

harborsync:
  redis:
    drone-state-ttl-seconds: 30    # Her drone kaydı 30 saniye sonra expire olur
```

```
queues:
  telemetry-processed: telemetry.processed
```

DTO'lar

```
// dto/RawTelemetryPayload.java – Drone simülatöründen gelen ham veri
@Data @NoArgsConstructor
public class RawTelemetryPayload {
    @NotBlank private String droneId;
    @NotBlank private String sector;
    @Min(0) private int containerCount;
    @Min(1) private int capacity;
    private boolean blockageDetected;
    @NotBlank private String timestamp;
    private String vesselEta; // Opsiyonel – yaklaşan gemi varsa dolu
}

// dto/ProcessedTelemetryEvent.java – Kuyruğa basılan event
@Data @Builder
public class ProcessedTelemetryEvent {
    private String correlationId;
    private String sector;
    private double fillRate; // containerCount / capacity
    private boolean blockageDetected;
    private String droneId;
    private String vesselEta;
    private String timestamp;
}
```

Redis Konfigürasyonu

```
// config/RedisConfig.java
@Configuration
public class RedisConfig {

    @Bean
    public RedisTemplate<String, String> redisTemplate(RedisConnectionFactory factory) {
        RedisTemplate<String, String> template = new RedisTemplate<>();
        template.setConnectionFactory(factory);
        template.setKeySerializer(new StringRedisSerializer());
        template.setValueSerializer(new StringRedisSerializer());
        return template;
    }
}
```


Telemetry Service Katmanı

```
// service/TelemetryService.java
@Service
@RequiredArgsConstructor
@Slf4j
public class TelemetryService {

    private final RedisTemplate<String, String> redisTemplate;
    private final TelemetryProducer telemetryProducer;
    private final ObjectMapper objectMapper;

    @Value("${harborsync.redis.drone-state-ttl-seconds}")
    private long droneTtlSeconds;

    public void process(RawTelemetryPayload payload, String correlationId) {
        // 1. Doğrulama – capacity 0 olamaz (ZeroDivisionError önlemi)
        if (payload.getCapacity() <= 0) {
            throw new IllegalArgumentException("Capacity must be > 0 for drone " + payload.getDroneId());
        }

        // 2. Doluluk oranını hesapla
        double fillRate = (double) payload.getContainerCount() / payload.getCapacity();

        // 3. Son drone state'ini Redis'e yaz (TTL: 30 saniye)
        String redisKey = "drone:" + payload.getDroneId();
        try {
            String stateJson = objectMapper.writeValueAsString(payload);
            redisTemplate.opsForValue().set(redisKey, stateJson, droneTtlSeconds, TimeUnit.SECONDS);
        } catch (JsonProcessingException e) {
            log.warn("[{}] Redis write failed for drone {}: {}", correlationId, payload.getDroneId(), e);
        }

        // 4. İşlenmiş event'i RabbitMQ'ya yayınla
        ProcessedTelemetryEvent event = ProcessedTelemetryEvent.builder()
            .correlationId(correlationId)
            .sector(payload.getSector())
            .fillRate(fillRate)
            .blockageDetected(payload.isBlockageDetected())
            .droneId(payload.getDroneId())
            .vesselEta(payload.getVesselEta())
            .timestamp(payload.getTimestamp())
            .build();

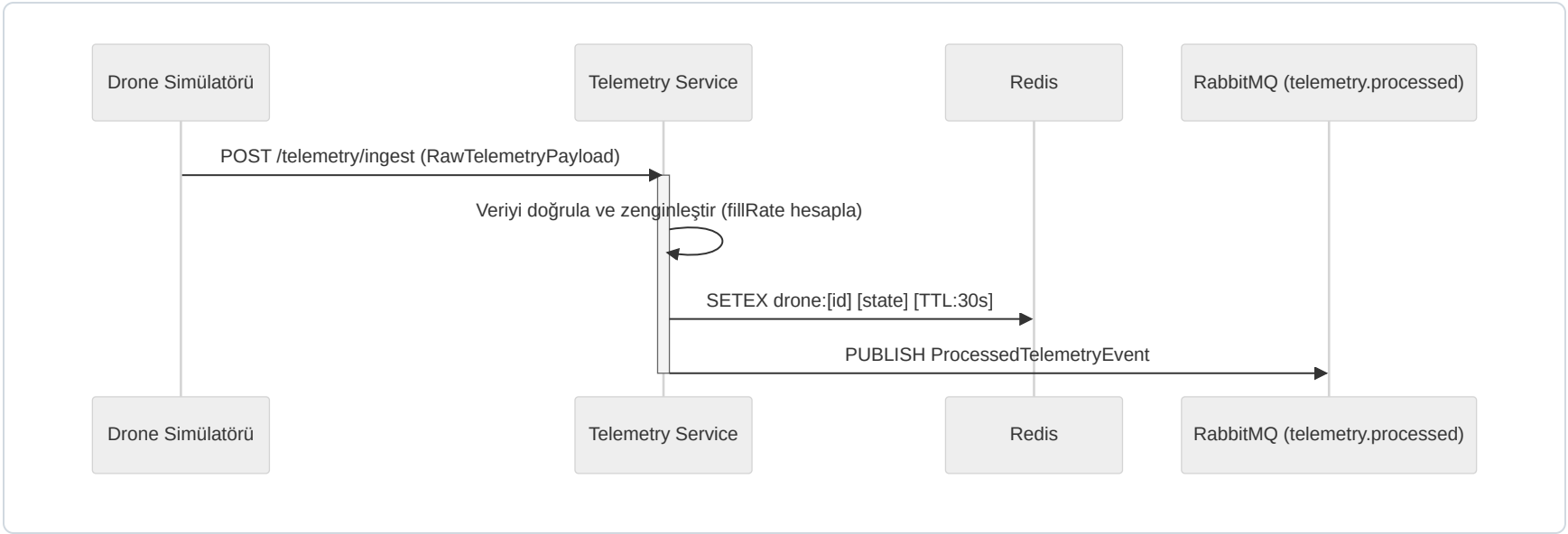
        telemetryProducer.publish(event);
        log.info("[{}] Telemetry processed: sector={} fillRate={}", correlationId, payload.getSector(),
```

```
}  
}  
}
```

REST Controller

```
// controller/TelemetryController.java  
@RestController  
@RequestMapping("/telemetry")  
@RequiredArgsConstructor  
public class TelemetryController {  
  
    private final TelemetryService telemetryService;  
  
    @PostMapping("/ingest")  
    @ResponseStatus(HttpStatus.ACCEPTED)  
    public void ingest(  
        @Valid @RequestBody RawTelemetryPayload payload,  
        @RequestHeader(value = "X-Correlation-ID", required = false) String correlationId) {  
        if (correlationId == null) correlationId = UUID.randomUUID().toString();  
        telemetryService.process(payload, correlationId);  
    }  
}
```

İş Akışı Diyagramı



Drone Simülâtörü (Python Script)

Dosya: drone-simulator/simulate.py

```
import requests
import json
import time
import random
from datetime import datetime

TELEMETRY_URL = "http://localhost:8082/telemetry/ingest"
SECTORS = ["A-01", "B-12", "C-07", "D-04"]
DRONES = ["HD-01", "HD-07", "HD-11"]

def generate_telemetry():
    sector = random.choice(SECTORS)
    capacity = 100
    container_count = random.randint(70, 100)
    return {
        "droneId": random.choice(DRONES),
        "sector": sector,
        "containerCount": container_count,
        "capacity": capacity,
        "blockageDetected": random.random() < 0.15,    # %15 ihtimalle tıkanıklık
        "timestamp": datetime.utcnow().isoformat(),
        "vesselEta": "14:30" if random.random() < 0.3 else None
    }

if __name__ == "__main__":
    print("Drone simulator started. Press Ctrl+C to stop.")
    while True:
        payload = generate_telemetry()
        try:
            r = requests.post(TELEMETRY_URL, json=payload, timeout=2)
            print(f"[{payload['droneId']}] sector={payload['sector']} "
                  f"fill={payload['containerCount']}% → {r.status_code}")
        except Exception as e:
            print(f"Simulator error: {e}")
        time.sleep(2)
```

4.5 Task Assignment Service

Sorumlu: Deniz Yıldız · Port: 8084

`congestion.alert` kuyruğunu dinler, Vessel Service'ten gemi durumunu sorgular ve vinç/ekip görev ataması yapar. **Bu servis en karmaşık servistir; Saga Pattern ve Circuit Breaker içerir.**

Teknoloji Yığını

Katman	Teknoloji	Notlar
Framework	Spring Boot 3.2.x	Ana uygulama çatısı
Veri Erişimi	Spring Data JPA	Görev kayıtları için PostgreSQL
Dayanıklılık	Resilience4j	Vessel Service çağrıları için Circuit Breaker
HTTP İstemcisi	Spring WebClient	Vessel Service'e reaktif çağrılar
Mesajlaşma	Spring AMQP	Alarm dinleme ve görev olayı yayınlama
Port	8084	—

Maven pom.xml Bağımlılıkları

```
<dependencies>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-web</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-jpa</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-amqp</artifactId>
  </dependency>
  <dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-webflux</artifactId>
    <!-- WebClient için -->
  </dependency>
  <dependency>
    <groupId>org.postgresql</groupId>
    <artifactId>postgresql</artifactId>
    <scope>runtime</scope>
  </dependency>
  <dependency>
    <groupId>io.github.resilience4j</groupId>
    <artifactId>resilience4j-spring-boot3</artifactId>
  </dependency>
</dependencies>
```

```
        <groupId>org.flywaydb</groupId>
        <artifactId>flyway-core</artifactId>
    </dependency>
    <dependency>
        <groupId>org.projectlombok</groupId>
        <artifactId>lombok</artifactId>
        <optional>true</optional>
    </dependency>
</dependencies>
```

Veritabanı Şeması — Flyway Migration

Dosya: src/main/resources/db/migration/V1__create_tasks_table.sql

```
CREATE TABLE tasks (
    id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    sector      VARCHAR(20) NOT NULL,
    alert_type  VARCHAR(30) NOT NULL,
    assigned_unit VARCHAR(50) NOT NULL,
    priority    VARCHAR(10) NOT NULL,
    status      VARCHAR(20) NOT NULL DEFAULT 'PENDING',
    correlation_id VARCHAR(100),
    created_at  TIMESTAMP NOT NULL DEFAULT NOW(),
    completed_at  TIMESTAMP
);

ALTER TABLE tasks ADD CONSTRAINT chk_task_status
    CHECK (status IN ('PENDING', 'IN_PROGRESS', 'COMPLETED', 'FAILED'));

ALTER TABLE tasks ADD CONSTRAINT chk_task_priority
    CHECK (priority IN ('HIGH', 'MEDIUM', 'LOW'));

CREATE INDEX idx_task_status ON tasks(status);
CREATE INDEX idx_task_sector ON tasks(sector);
```

application.yml (Resilience4j dahil)

```
server:
  port: 8084

spring:
  application:
    name: task-assignment-service
  datasource:
    url: jdbc:postgresql://localhost:5434/task_db
```



```
username: harbor
password: harbor123
rabbitmq:
  host: localhost
  port: 5672
  username: guest
  password: guest

vessel:
  service:
    url: http://vessel-service:8081

resilience4j:
  circuitbreaker:
    instances:
      vessel-service:
        register-health-indicator: true
        sliding-window-size: 5
        minimum-number-of-calls: 3
        failure-rate-threshold: 50          # %50 hata → devre açılır
        wait-duration-in-open-state: 10s
        permitted-number-of-calls-in-half-open-state: 2
```

Anti-Corruption Layer — VesselServiceClient

```
// client/VesselServiceClient.java
@Component
@RequiredArgsConstructor
@Slf4j
public class VesselServiceClient {

    private final WebClient webClient;

    @CircuitBreaker(name = "vessel-service", fallbackMethod = "getArrivingVesselsFallback")
    public List<VesselResponse> getArrivingVessels() {
        return webClient.get()
            .uri("/vessels?status=ARRIVING")
            .retrieve()
            .bodyToFlux(VesselResponse.class)
            .collectList()
            .block();
    }

    /**
     * Circuit Breaker açıksa boş liste döner.
     * Production'da Redis cache kullanılır.
     */
}
```

```
        public List<VesselResponse> getArrivingVesselsFallback(Throwable t) {
            log.warn("Vessel Service unavailable, using fallback. Reason: {}", t.getMessage());
            return List.of();
        }
    }

// config/WebClientConfig.java
@Configuration
public class WebClientConfig {

    @Bean
    public WebClient vesselServiceClient(
        @Value("${vessel.service.url:http://vessel-service:8081}") String baseUrl) {
        return WebClient.builder().baseUrl(baseUrl).build();
    }
}
```

Task Assignment Consumer — Saga Pattern

```
// messaging/consumer/CongestionAlertConsumer.java
@Component
@RequiredArgsConstructor
@Slf4j
public class CongestionAlertConsumer {

    private final TaskAssignmentService taskService;

    @RabbitListener(queues = "congestion.alert")
    public void onAlertReceived(CongestionAlertEvent alert) {
        MDC.put("correlationId", alert.getCorrelationId());
        try {
            log.info("[{}] Alert received: {} for sector {}",
                alert.getCorrelationId(), alert.getAlertType(), alert.getSector());
            taskService.handleAlert(alert);
        } finally {
            MDC.clear();
        }
    }
}

// service/TaskAssignmentService.java
@Service
@RequiredArgsConstructor
@Slf4j
@Transactional
public class TaskAssignmentService {
```

```
private final TaskRepository taskRepository;
private final VesselServiceClient vesselClient;
private final TaskCreatedProducer taskCreatedProducer;

/**
 * Saga adımları:
 * 1. Vessel Service'ten gelen gemileri sorgula (Circuit Breaker korumalı)
 * 2. Görevi oluştur ve kaydet
 * 3. task.created event'i yayınla
 * Herhangi bir adımda hata → görev FAILED olarak kaydedilir (compensating transaction)
 */
public void handleAlert(CongestionAlertEvent alert) {
    // Saga Adım 1: Mevcut gemileri sorgula
    List<VesselResponse> arrivingVessels = vesselClient.getArrivingVessels();
    String assignedUnit = determineAssignedUnit(alert, arrivingVessels);

    // Saga Adım 2: Görevi kaydet
    Task task = new Task();
    task.setSector(alert.getSector());
    task.setAlertType(alert.getAlertType());
    task.setAssignedUnit(assignedUnit);
    task.setPriority(alert.getSeverity());
    task.setStatus(TaskStatus.PENDING);
    task.setCorrelationId(alert.getCorrelationId());

    Task savedTask = taskRepository.save(task);
    log.info("[{}] Task created: id={} unit={} sector={}",
        alert.getCorrelationId(), savedTask.getId(), assignedUnit, alert.getSector());

    // Saga Adım 3: Event yayınla
    taskCreatedProducer.publish(TaskCreatedEvent.builder()
        .correlationId(alert.getCorrelationId())
        .taskId(savedTask.getId())
        .sector(alert.getSector())
        .assignedUnit(assignedUnit)
        .priority(alert.getSeverity())
        .build());
}

private String determineAssignedUnit(CongestionAlertEvent alert, List<VesselResponse> vessels) {
    return switch (alert.getAlertType()) {
        case "SECTOR_CRITICAL", "IMMEDIATE_ACTION" -> "Crane-" + (int)(Math.random() * 5 + 1);
        default -> "Team-" + (int)(Math.random() * 3 + 1);
    };
}
}
```



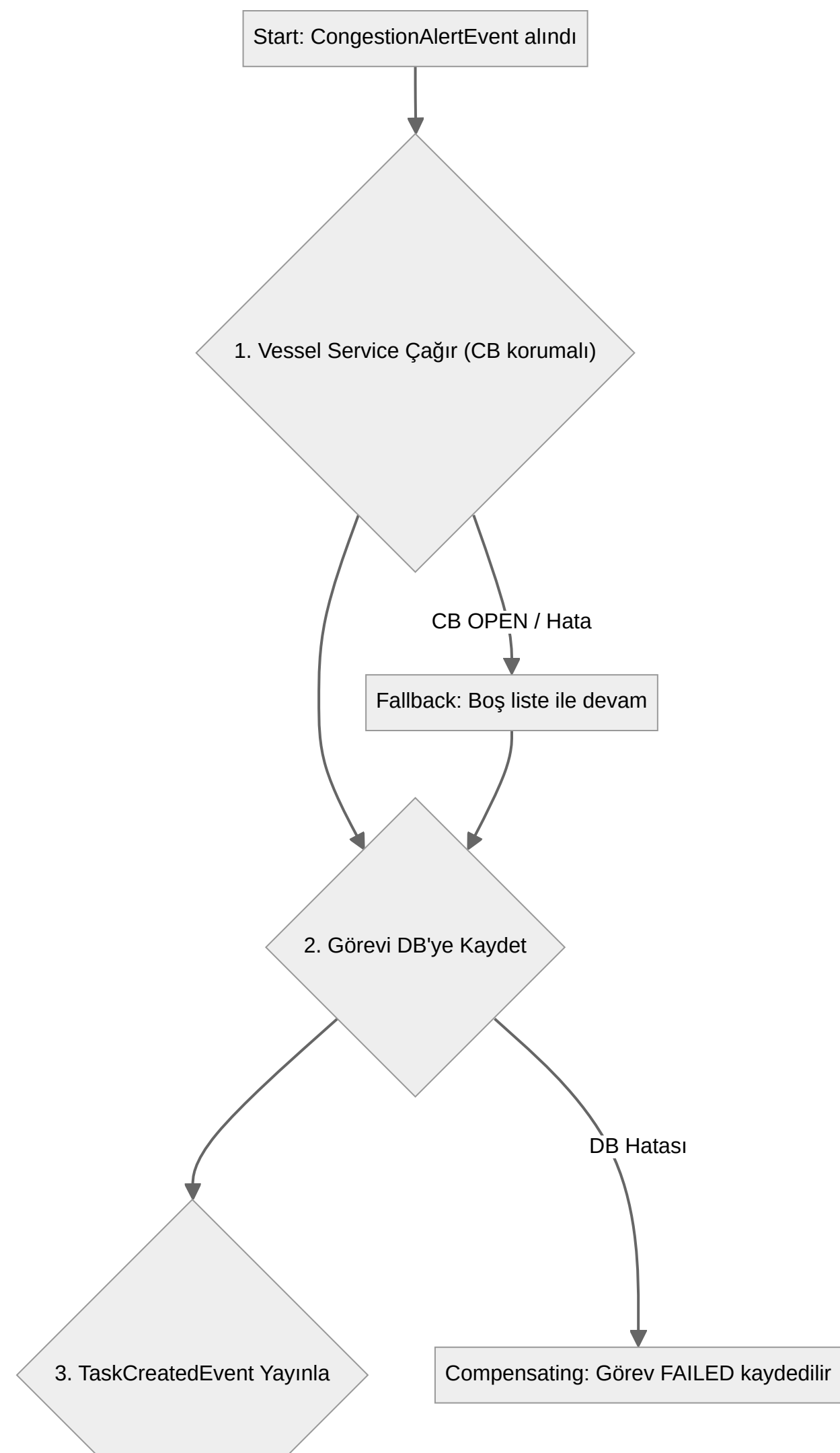
```
// controller/TaskController.java
@RestController
@RequestMapping("/tasks")
@RequiredArgsConstructor
public class TaskController {

    private final TaskRepository taskRepository;

    @GetMapping("/pending")
    public List<Task> getPendingTasks() {
        return taskRepository.findByStatus(TaskStatus.PENDING);
    }

    @PutMapping("/{id}/complete")
    public Task completeTask(@PathVariable UUID id) {
        Task task = taskRepository.findById(id)
            .orElseThrow(() -> new RuntimeException("Task not found: " + id));
        task.setStatus(TaskStatus.COMPLETED);
        task.setCompletedAt(LocalDateTime.now());
        return taskRepository.save(task);
    }
}
```

Saga ve Circuit Breaker Akış Diyagramı



End: Başarılı

Circuit Breaker Durumları: CLOSED (normal) → hata oranı %50'yi aşarsa OPEN → 10s sonra HALF-OPEN → başarılı denemelerle CLOSED. OPEN durumunda fallback metodu çalışır, servis çalışmaya devam eder.

4.6 Notification Service

Sorumlu: Deniz Yıldız · Port: 8085

`congestion.alert`, `task.created` ve `dlq.errors` kuyruklarını dinler; tüm olayları yapılandırılmış log formatında `alerts.log` dosyasına yazar. REST endpoint gerekmez. FastAPI (Python) ile geliştirilmiştir — mimarinin poliglot yapısını gösteren bir örnektir.

Teknoloji Yığını

Katman	Teknoloji	Versiyon
Framework	FastAPI	0.110.x
Python	3.11	—
RabbitMQ Client	aio-pika (async)	9.x
Log	Python logging + RotatingFileHandler	—
Port	8085	—

Proje Yapısı

```
notification-service/  
├─ Dockerfile  
├─ requirements.txt
```

```
├─ main.py
├─ config.py
├─ consumer.py
├─ formatter.py
├─ logs/
│   └─ alerts.log
```

requirements.txt

```
fastapi==0.110.0
uvicorn==0.29.0
aio-pika==9.4.1
pydantic==2.6.4
python-json-logger==2.0.7
```

config.py

```
import os

RABBITMQ_URL = os.getenv("RABBITMQ_URL", "amqp://guest:guest@localhost:5672/")

QUEUES = {
    "congestion_alert": "congestion.alert",
    "task_created": "task.created",
    "dlq": "dlq.errors",
}

LOG_FILE = "logs/alerts.log"
```

formatter.py — Yapılandırılmış Log

```
import logging
import logging.handlers
from config import LOG_FILE
import os

os.makedirs("logs", exist_ok=True)

def get_logger(name: str) -> logging.Logger:
    logger = logging.getLogger(name)
    logger.setLevel(logging.DEBUG)

    # Dosya handler – 10MB, 5 yedek
```



```
        file_handler = logging.handlers.RotatingFileHandler(
            LOG_FILE, maxBytes=10_000_000, backupCount=5
        )
        file_handler.setLevel(logging.DEBUG)

        # Console handler
        console_handler = logging.StreamHandler()
        console_handler.setLevel(logging.INFO)

        fmt = "[% (asctime)s] [% (levelname)-8s] [% (correlation_id)s] %(message)s"
        formatter = logging.Formatter(fmt, datefmt="%Y-%m-%d %H:%M:%S")
        file_handler.setFormatter(formatter)
        console_handler.setFormatter(formatter)

        logger.addHandler(file_handler)
        logger.addHandler(console_handler)
        return logger
```

consumer.py — Async RabbitMQ Consumer

```
import json
import aio_pika
from formatter import get_logger
from config import RABBITMQ_URL, QUEUES

logger = get_logger("notification")

def _log(level: str, correlation_id: str, message: str):
    extra = {"correlation_id": correlation_id or "N/A"}
    getattr(logger, level)(message, extra=extra)

async def handle_congestion_alert(message: aio_pika.IncomingMessage):
    async with message.process():
        try:
            payload = json.loads(message.body)
            corr_id = payload.get("correlationId", "N/A")
            alert_type = payload.get("alertType", "UNKNOWN")
            sector = payload.get("sector", "?")
            fill_rate = payload.get("fillRate", 0)
            _log("warning", corr_id,
                f"[{alert_type}] Sector {sector} at {fill_rate*100:.0f}% capacity")
        except Exception as e:
            logger.error(f"Failed to process congestion.alert: {e}",
                extra={"correlation_id": "ERR"})
```

```

async def handle_task_created(message: aio_pika.IncomingMessage):
    async with message.process():
        try:
            payload = json.loads(message.body)
            corr_id = payload.get("correlationId", "N/A")
            unit = payload.get("assignedUnit", "?")
            sector = payload.get("sector", "?")
            priority = payload.get("priority", "?")
            _log("info", corr_id,
                f"[TASK] {unit} assigned to {sector} (priority: {priority})")
        except Exception as e:
            logger.error(f"Failed to process task.created: {e}",
                extra={"correlation_id": "ERR"})

async def handle_dlq(message: aio_pika.IncomingMessage):
    async with message.process():
        msg_id = message.message_id or "unknown"
        _log("error", "DLQ", f"Failed message requeued: msg-id={msg_id}")

async def start_consumers():
    connection = await aio_pika.connect_robust(RABBITMQ_URL)
    channel = await connection.channel()
    await channel.set_qos(prefetch_count=10)

    congestion_q = await channel.declare_queue(QUEUES["congestion_alert"], durable=True)
    task_q = await channel.declare_queue(QUEUES["task_created"], durable=True)
    dlq_q = await channel.declare_queue(QUEUES["dlq"], durable=True)

    await congestion_q.consume(handle_congestion_alert)
    await task_q.consume(handle_task_created)
    await dlq_q.consume(handle_dlq)

    logger.info("Notification Service consumers started.", extra={"correlation_id": "INIT"})
    return connection

```

main.py

```

import asyncio
from contextlib import asynccontextmanager
from fastapi import FastAPI
from consumer import start_consumers
from formatter import get_logger

logger = get_logger("notification.main")

```

```
@asynccontextmanager
async def lifespan(app: FastAPI):
    # Uygulama başlarken consumer'ları başlat
    connection = await start_consumers()
    yield
    # Uygulama kapanırken bağlantıyı kapat
    await connection.close()

app = FastAPI(title="HarborSync Notification Service", lifespan=lifespan)

@app.get("/health")
def health():
    return {"status": "UP", "service": "notification-service"}
```

Dockerfile

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN mkdir -p logs

CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8085"]
```

5. API Sözleşmeleri ve Olay Şemaları

REST API Sözleşmeleri

Servis	Metot & Yol	Açıklama
Vessel Service	POST /api/vessels	Yeni gemi kaydı oluşturur
Vessel Service	GET /api/vessels?status=ARRIVING	Duruma göre filtreli gemi listesi
Vessel Service	GET /api/vessels/{id}	Gemi detayı
Vessel Service	PUT /api/vessels/{id}/status	Durum ve rıhtım güncelle
Telemetry Service	POST /telemetry/ingest	Ham telemetri alımı (drone'dan)
Task Assignment	GET /api/tasks/pending	Bekleyen görevleri listele
Task Assignment	PUT /api/tasks/{id}/complete	Görevi tamamlandı olarak işaretle

RabbitMQ Olay Şemaları

telemetry.processed — Telemetry Service → Congestion Analysis

```
{
  "correlationId": "OP-2025-001",
  "sector": "B-12",
  "fillRate": 0.94,
  "blockageDetected": true,
  "droneId": "HD-07",
  "vesselEta": "14:30",
  "timestamp": "2025-01-15T14:28:00Z"
}
```

congestion.alert — Congestion Analysis → Task Assignment & Notification

```
{
  "correlationId": "OP-2025-001",
  "alertType": "SECTOR_CRITICAL",
  "sector": "B-12",
  "severity": "HIGH",
  "fillRate": 0.94,
  "recommendedAction": "REDIRECT_CRANE",
  "timestamp": "2025-01-15T14:28:03Z"
}
```

task.created — Task Assignment → Notification

```
{
  "correlationId": "0P-2025-001",
  "taskId": "550e8400-e29b-41d4-a716",
  "sector": "B-12",
  "assignedUnit": "Crane-3",
  "priority": "HIGH",
  "timestamp": "2025-01-15T14:28:04Z"
}
```

6. Veritabanı Mimarisi

HarborSync, "Database per Service" desenini benimser. Her servis kendi veritabanına sahiptir ve başka servislerin veritabanına doğrudan erişmez.

Vessel Service — PostgreSQL (vessel_db)

Vessels		
UUID	id	PK
VARCHAR	name	
VARCHAR	imo_number	UK
VARCHAR	status	
VARCHAR	berth	
TIMESTAMP	eta	
TIMESTAMP	created_at	
TIMESTAMP	updated_at	

Task Assignment Service — PostgreSQL (task_db)

Tasks		
UUID	id	PK
VARCHAR	sector	
VARCHAR	alert_type	
VARCHAR	assigned_unit	
VARCHAR	priority	
VARCHAR	status	
VARCHAR	correlation_id	
TIMESTAMP	created_at	
TIMESTAMP	completed_at	

Telemetry Service — Redis

Özellik	Değer
Teknoloji	Redis 7
Anahtar Yapısı	drone:[droneId] (örn: drone:HD-07)
Değer	Ham telemetri verisinin JSON string hali
TTL	30 saniye (yapılandırılabilir)
Amaç	Drone'ların son bilinen durumunu geçici olarak saklamak

7. Frontend Mimarisi

Bu projede kullanıcı arayüzü (Frontend) bulunmamaktadır.

HarborSync, "headless" bir sistem olarak tasarlanmıştır. Tüm işlevselliği REST API'ler ve asenkron olaylar üzerinden sunulur. Çıktılar görev atamaları ve Notification Service tarafından üretilen yapılandırılmış loglardır. Bu yapı, gelecekte bir web arayüzü veya mobil uygulamanın bu API'leri kullanarak entegre olmasına olanak tanır.

8. DevOps ve Altyapı

Tüm servisler ve bağımlılıklar tek bir `docker-compose up --build` komutu ile ayağa kalkar.

docker-compose.yml (Tam Yapı)

```
version: '3.8'

services:
  rabbitmq:
    image: rabbitmq:3.13-management
    container_name: harborsync-rabbitmq
    ports:
      - "5672:5672"          # AMQP
      - "15672:15672"       # Management UI → http://localhost:15672
    environment:
      RABBITMQ_DEFAULT_USER: guest
      RABBITMQ_DEFAULT_PASS: guest
    healthcheck:
      test: ["CMD", "rabbitmq-diagnostics", "ping"]
      interval: 10s
      timeout: 5s
      retries: 5

  postgres-vessel:
    image: postgres:15
    container_name: vessel-db
    environment:
      POSTGRES_DB: vessel_db
      POSTGRES_USER: harbor
      POSTGRES_PASSWORD: harbor123
    ports:
      - "5433:5432"

  postgres-task:
    image: postgres:15
    container_name: task-db
    environment:
      POSTGRES_DB: task_db
      POSTGRES_USER: harbor
      POSTGRES_PASSWORD: harbor123
    ports:
      - "5434:5432"

  redis:
    image: redis:7-alpine
```

```
container_name: harborsync-redis
ports:
  - "6379:6379"
command: redis-server --save "" --appendonly no

vessel-service:
  build: ./vessel-service
  container_name: vessel-service
  ports:
    - "8081:8081"
  depends_on:
    postgres-vessel:
      condition: service_started
    rabbitmq:
      condition: service_healthy
  environment:
    SPRING_DATASOURCE_URL: jdbc:postgresql://vessel-db:5432/vessel_db
    SPRING_RABBITMQ_HOST: rabbitmq

telemetry-service:
  build: ./telemetry-service
  container_name: telemetry-service
  ports:
    - "8082:8082"
  depends_on:
    - redis
    - rabbitmq
  environment:
    SPRING_REDIS_HOST: redis
    SPRING_RABBITMQ_HOST: rabbitmq

congestion-analysis:
  build: ./congestion-analysis
  container_name: congestion-analysis
  ports:
    - "8083:8083"
  depends_on:
    rabbitmq:
      condition: service_healthy

task-assignment-service:
  build: ./task-assignment-service
  container_name: task-assignment-service
  ports:
    - "8084:8084"
  depends_on:
    - postgres-task
    - rabbitmq
  environment:
```



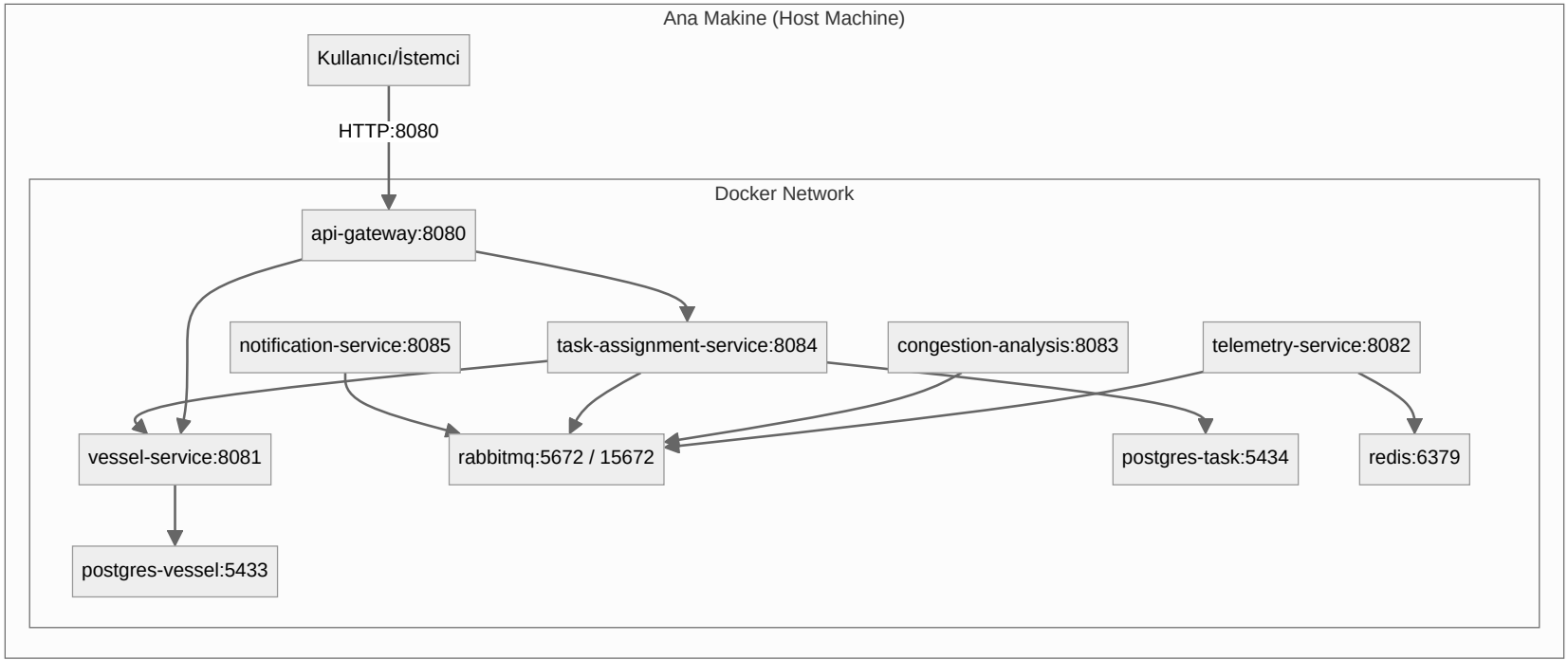
```

    SPRING_DATASOURCE_URL: jdbc:postgresql://task-db:5432/task_db
    VESSEL_SERVICE_URL: http://vessel-service:8081

notification-service:
  build: ./notification-service
  container_name: notification-service
  ports:
    - "8085:8085"
  depends_on:
    - rabbitmq

gateway:
  build: ./gateway
  container_name: api-gateway
  ports:
    - "8080:8080"
  depends_on:
    - vessel-service
    - task-assignment-service
  environment:
    VESSEL_SERVICE_URL: http://vessel-service:8081
    TASK_SERVICE_URL: http://task-assignment-service:8084
```

Dağıtım Topolojisi



9. Güvenlik Mimarisi

Mevcut durumda güvenlik temel seviyededir; altyapı izolasyonu ve merkezi giriş noktası kontrolüne odaklanılmıştır.

Katman	Mevcut Durum	Notlar
Kimlik Doğrulama	Yok	Tüm endpoint'ler halka açık
Yetkilendirme	Yok	Rol bazlı erişim kontrolü yok
Ağ İzolasyonu	Docker internal network	Servisler dış dünyaya kapalı
Rate Limiting	API Gateway (Redis tabanlı)	10 req/s normal, 20 req/s burst
Sır Yönetimi	docker-compose env vars	Düz metin — üretim için uygun değil

Kritik Güvenlik Açığı: Kimlik doğrulama ve yetkilendirme mekanizmaları bulunmamaktadır. Üretim ortamına geçiş öncesinde OAuth2/JWT tabanlı bir auth katmanı API Gateway üzerine eklenmesi zorunludur.

10. Gözlemlenebilirlik ve İzleme

Yapılandırılmış ve Merkezi Loglama

Notification Service , sistemdeki tüm önemli iş olaylarını tek bir yerde toplar. Her log satırı zaman damgası, seviye, mesaj ve **correlation ID** içerir.

```
[14:28:03] [WARNING ] [OP-XXXXXXX] [SECTOR_CRITICAL] Sector B-12 at 94% capacity
[14:28:04] [INFO    ] [OP-XXXXXXX] [TASK] Crane-3 assigned to B-12 (priority: HIGH)
```

Correlation ID'nin Rolü: API Gateway tarafından her isteğe atanan X-Correlation-ID , tüm iç işlemlere (servis çağrıları, kuyruk mesajları) taşınır. Log dosyasında bu ID filtrelenerek bir isteğin tüm yolculuğu tek seferde görülebilir.

Sağlık Kontrolleri

Her Spring Boot servisi `spring-boot-starter-actuator` sayesinde `/actuator/health` endpoint'ini sunar. Docker Compose'da RabbitMQ için `healthcheck` tanımlıdır; servisler RabbitMQ sağlıklı olmadan başlamaz.

Eksik Gözlemlenebilirlik Araçları

Araç	Amaç	Durum
Prometheus	Metrik toplama	Entegre değil
Grafana	Metrik görselleştirme	Entegre değil
Jaeger / Zipkin	Dağıtık izleme (tracing)	Entegre değil
ELK Stack	Log aggregation	Entegre değil

11. Riskler ve Teknik Borç

Güvenlik Açıkları

Risk: Kimlik doğrulama ve yetkilendirme mekanizmaları yoktur. Tüm API endpoint'leri halka açıktır.
Etki: Kötü niyetli kullanıcılar sistemi manipüle edebilir, sahte kayıtlar oluşturabilir, sistemi aşırı yükleyebilir.

Tekil Hata Noktaları (SPOF)

Risk: RabbitMQ, API Gateway ve veritabanları tek birer instance olarak çalışmaktadır.
Etki: Bu bileşenlerden herhangi birinin çökmesi tüm sistemin durmasına neden olur. Geliştirme ortamı için kabul edilebilir, üretim için yüksek risk.

Eksik Gözlemlenebilirlik Araçları

Risk: Prometheus, Grafana, Jaeger gibi modern gözlemlenebilirlik araçları entegre değildir.

Etki: Sorunları proaktif tespit yerine reaktif müdahale zorunluluğu. Performans darboğazlarını tespit etmek zordur.

Yapılandırma ve Sır Yönetimi

Risk: Veritabanı şifreleri `docker-compose.yml` ve `application.yml` dosyalarında düz metin olarak saklanmaktadır.

Etki: Kod tabanına erişimi olan herkesin bu sırlara da erişimi olur.

Task Assignment — Görev Atama Mantiğı

Risk: `determineAssignedUnit()` metodu rastgele bir birim atar; gerçek kaynak havuzu yönetimi yoktur.

Etki: Aynı birime birden fazla görev atanabilir, kaynak çakışmaları oluşabilir.

12. İyileştirme Önerileri

1. Güvenlik Katmanını Güçlendirme

- Kimlik Doğrulama:** API Gateway üzerinde OAuth2/JWT tabanlı auth akışı implemente edilmelidir.
- Yetkilendirme:** Roller ve yetkiler (scopes) tanımlanarak belirli endpoint'lere sadece yetkili erişim sağlanmalıdır.
- Sır Yönetimi:** HashiCorp Vault veya AWS Secrets Manager kullanılarak sırlar kod tabanından çıkarılmalıdır.

2. Yüksek Erişilebilirlik (HA) için Altyapı İyileştirme

- RabbitMQ:** Üretimde en az 3 node'dan oluşan bir küme kurulmalıdır.
- Servis Replikasyonu:** Stateless servisler Kubernetes üzerinde en az 2-3 replika ile çalıştırılmalıdır.
- Veritabanı:** Yönetilen veritabanı servisleri (AWS RDS, Azure PostgreSQL) veya primary-replica replikasyon kullanılmalıdır.

3. Tam Kapsamlı Gözlemlenebilirlik Yığını

- **Metrikler:** Micrometer + Prometheus + Grafana entegrasyonu ile istek sayısı, gecikme, hata oranları izlenmelidir.
- **Dağıtık İzleme:** OpenTelemetry SDK'ları ile Jaeger veya Zipkin entegrasyonu yapılmalıdır.
- **Log Aggregation:** ELK Stack (Elasticsearch, Logstash, Kibana) veya Loki + Grafana ile merkezi log yönetimi sağlanmalıdır.

4. Merkezi Yapılandırma Yönetimi

- Spring Cloud Config Server veya HashiCorp Consul ile yapılandırma dosyaları kod tabanından ayrılmalıdır.

13. Test Stratejisi

Her servis için birim (unit) ve entegrasyon testlerini kapsayan katmanlı bir test yaklaşımı benimser.

Minimum Test Kapsamı (Her Servis)

```
src/test/
├─ unit/
│   └─ service/           # İş mantığı testleri (mock kullanılır)
└─ integration/
    └─ messaging/         # RabbitMQ ile entegrasyon (Testcontainers)
```

Birim Test Örneği — Congestion Analysis Kural Motoru

```
@ExtendWith(MockitoExtension.class)
class CongestionRuleEngineTest {

    @InjectMocks
    CongestionRuleEngine ruleEngine;

    @BeforeEach
    void setUp() {
        ReflectionTestUtils.setField(ruleEngine, "criticalThreshold", 0.90);
        ReflectionTestUtils.setField(ruleEngine, "warningThreshold", 0.85);
    }

    @Test
    void whenFillRateAboveCritical_shouldReturnCriticalAlert() {
        var event = new ProcessedTelemetryEvent("CID-1", "B-12", 0.94, false, "HD-07", null, "...");
```

```
        var result = ruleEngine.evaluate(event);
        assertTrue(result.isPresent());
        assertEquals("SECTOR_CRITICAL", result.get().getAlertType());
    }

    @Test
    void whenFillRateBelowWarning_shouldReturnEmpty() {
        var event = new ProcessedTelemetryEvent("CID-2", "A-01", 0.80, false, "HD-01", null, "...");
        var result = ruleEngine.evaluate(event);
        assertTrue(result.isEmpty());
    }

    @Test
    void whenBlockageAndVesselArriving_shouldReturnImmediateAction() {
        var event = new ProcessedTelemetryEvent("CID-3", "C-07", 0.70, true, "HD-11", "14:30", "...");
        var result = ruleEngine.evaluate(event);
        assertTrue(result.isPresent());
        assertEquals("IMMEDIATE_ACTION", result.get().getAlertType());
    }
}
```

Manuel Demo Test Senaryosu (Uçtan Uca)

```
# 1. Sistemi ayağa kaldır
docker-compose up --build

# 2. Gemi kaydı oluştur
curl -X POST http://localhost:8080/api/vessels \
  -H "Content-Type: application/json" \
  -d '{"name":"MV-Ankara","imoNumber":"IM01234567","eta":"2025-01-15T14:00:00"}'

# 3. Kritik telemetri gönder (doluluk %94, tıkanıklık var)
curl -X POST http://localhost:8082/telemetry/ingest \
  -H "Content-Type: application/json" \
  -d '{"droneId":"HD-07","sector":"B-12","containerCount":94,"capacity":100,"blockageDetected":true,"time":1705329600000}'

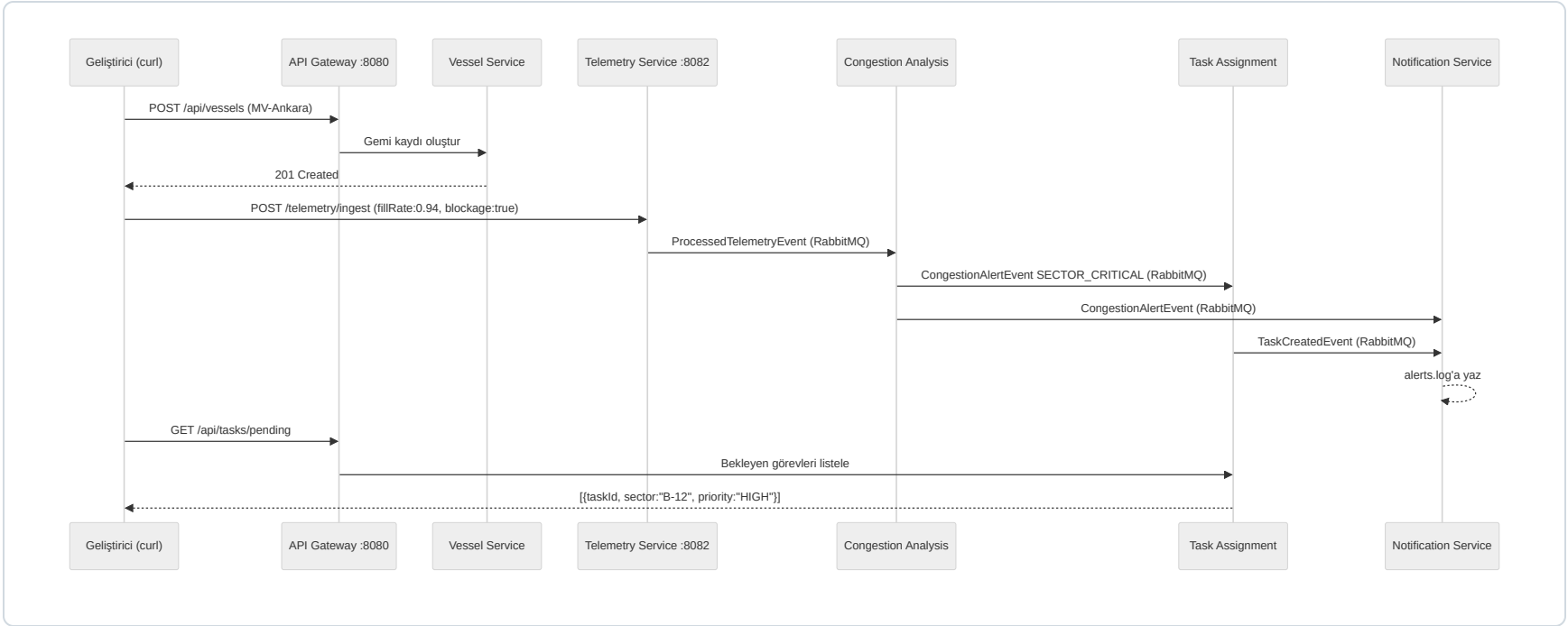
# 4. Görev oluşturuldu mu kontrol et
curl http://localhost:8080/api/tasks/pending

# 5. Notification loglarını kontrol et
docker logs harborsync-notification-service

# 6. RabbitMQ Management UI
# → http://localhost:15672 (guest/guest)
```



Uçtan Uca Akış Diyagramı



Beklenen Log Çıktısı:

```
[14:28:03] [WARNING ] [OP-XXXXXXX] [SECTOR_CRITICAL] Sector B-12 at 94% capacity
[14:28:04] [INFO    ] [OP-XXXXXXX] [TASK] Crane-3 assigned to B-12 (priority: HIGH)
```

14. Geliştirme Takvimi

Hafta	Emirhan Ersoy	Deniz Yıldız
Hafta 1	Vessel Service — DB şema + temel CRUD <i>Docker Compose altyapısı (ortak)</i>	Telemetry Service — Redis + ingest endpoint <i>Docker Compose altyapısı (ortak)</i>
Hafta 2	Congestion Analysis — RabbitMQ consumer/producer + kural motoru API Gateway — routing + Correlation ID filter	Task Assignment — DB şema + alert consumer Notification Service — consumer + log formatter
Hafta 3	Vessel Service unit testleri Congestion Analysis unit testleri	Task Assignment — Circuit Breaker + Saga Drone simülatörü

Hafta	Emirhan Ersoy	Deniz Yıldız
Hafta 4	End-to-end demo akışı testi README.md + mimari diyagram	End-to-end demo akışı testi README.md + mimari diyagram

15. Ekler

Teknoloji Yığını Özeti

Alan	Teknoloji	Kullanıldığı Servis(ler)
Backend Framework	Spring Boot 3.2.x	Vessel, Telemetry, Congestion, Task Assignment
Backend Framework	FastAPI (Python 3.11)	Notification Service
API Gateway	Spring Cloud Gateway 4.x	API Gateway
Veritabanı (İlişkisel)	PostgreSQL 15	Vessel Service, Task Assignment Service
Veritabanı (In-Memory)	Redis 7	Telemetry Service
Mesajlaşma	RabbitMQ 3.13	Tüm servisler
Dayanıklılık	Resilience4j (Circuit Breaker)	Task Assignment Service
Orkestrasyon	Docker Compose	Tüm sistem (yerel geliştirme)
Veritabanı Migration	Flyway	Vessel Service, Task Assignment Service
HTTP İstemcisi	Spring WebClient (Reactor)	Task Assignment Service
Build Aracı	Maven 3.9.x	Tüm Java servisleri

Terimler Sözlüğü

Terim	Açıklama
Eventual Consistency	Dağıtık sistemde tüm kopyaların belirli bir zaman sonunda aynı duruma gelmesini kabul eden tutarlılık modeli
Database per Service	Her mikroservisin kendi özel veritabanına sahip olduğu tasarım deseni
Circuit Breaker	Bağımlı servisteki hatanın kendi servisine yayılmasını engelleyen dayanıklılık deseni
Saga Pattern	Birden fazla servise yayılan işlemi yerel işlemler ve telafi edici olaylar dizisi olarak yöneten desen
Stateless	Servisin önceki etkileşimlere dair durum bilgisi saklamaması; yatay ölçeklemeyi kolaylaştırır
Correlation ID	Dağıtık sistemde bir isteğin tüm adımlarını takip etmeyi sağlayan benzersiz kimlik
DLQ (Dead Letter Queue)	İşlenemeyen mesajların yönlendirildiği özel kuyruk; mesaj kaybını önler
Anti-Corruption Layer	Bir servisi başka bir servisin iç modeline karşı izole eden adaptör katmanı
SPOF	Single Point of Failure — tek bir bileşenin çökmesinin tüm sistemi durdurduğu durum

Servis Bağımlılık Özeti

Servis	Bağımlı Olduğu Servisler	İletişim Tipi
API Gateway	Vessel Service, Task Assignment	HTTP (proxy)
Vessel Service	PostgreSQL	JDBC
Telemetry Service	Redis, RabbitMQ	Redis client, AMQP
Congestion Analysis	RabbitMQ	AMQP consumer/producer
Task Assignment	PostgreSQL, RabbitMQ, Vessel Service	JDBC, AMQP, HTTP (WebClient)
Notification Service	RabbitMQ	AMQP consumer

